



Welcome to the HPCIC AWS Tutorial Series!

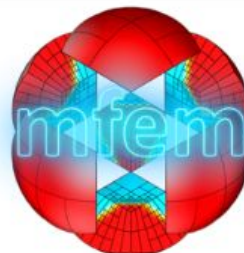
Go to:

<https://hpcic.llnl.gov/tutorials/2026-hpc-tutorials>

to learn more about our other
tutorials and documentation!



YGM





The Flux Framework Tutorial

HPCIC AWS Tutorial Series

July 21, 2026

Al Chu, James Corbett, Ryan Day, Jim Garlick, Mark Grondona, William Hobbs, Aniruddha Marathe, **Dan Milroy**, Zeke Morton, Chris Moussa, Tapasya Patki, Tom Scogland, **Vanessa Sochat**, Becky Springmeyer, Jae-Seung Yeom

Everything is *in flux*.

AI, cloud, and quantum are colliding with traditional HPC – and the ground keeps shifting under everyone building scientific workflows.



Simulation (HPC)

AI / ML

Cloud & Kubernetes

Quantum

We were preparing for this!

Flux is a resource manager designed to change as fast as the science does.

What is *Flux Framework*?

Flux Framework "Flux" is a system level **workload manager** developed by Lawrence Livermore National Laboratory that combines hierarchical **job management** with graph-based **scheduling**.

Did you know!

Flux has 25+ contributors, including principal engineers behind Slurm, and developers with expertise in container technologies, scheduling, and cloud computing.



Why is *Flux* a Framework?

Flux is composed of several, modular pieces that, when put together, result in the workload manager Flux. You can use these modules in other creative ways too.

flux-core

flux-sched

flux-security

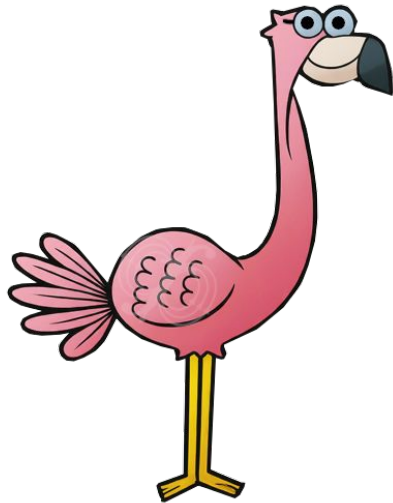
flux-accounting

flux-pmix



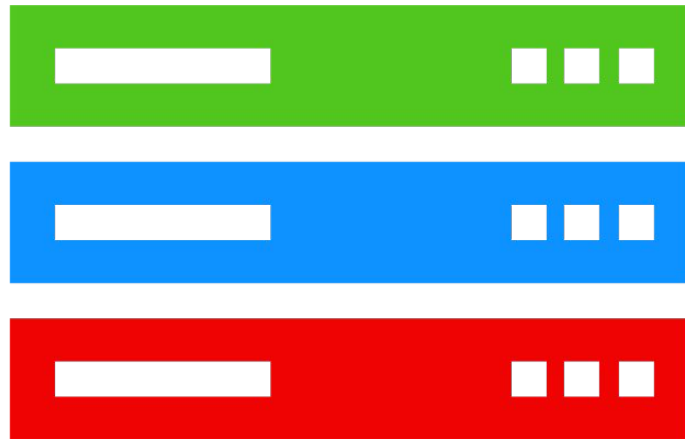
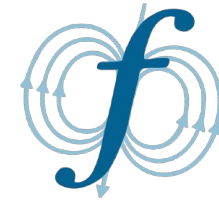
flux-framework

Flux is agnostic to where you run it



1. The system level scheduler of an HPC system
2. In user-space under another RM (e.g., Slurm)
3. As a single docker container
4. As an orchestrated set (e.g., docker-compose)
5. A full Flux cluster in Kubernetes (across clouds)
6. Virtual Machine setups (also across clouds)

Flux is part of the High Performance
Software Foundation



HPSF

HIGH PERFORMANCE
SOFTWARE FOUNDATION

How did Flux come to be?



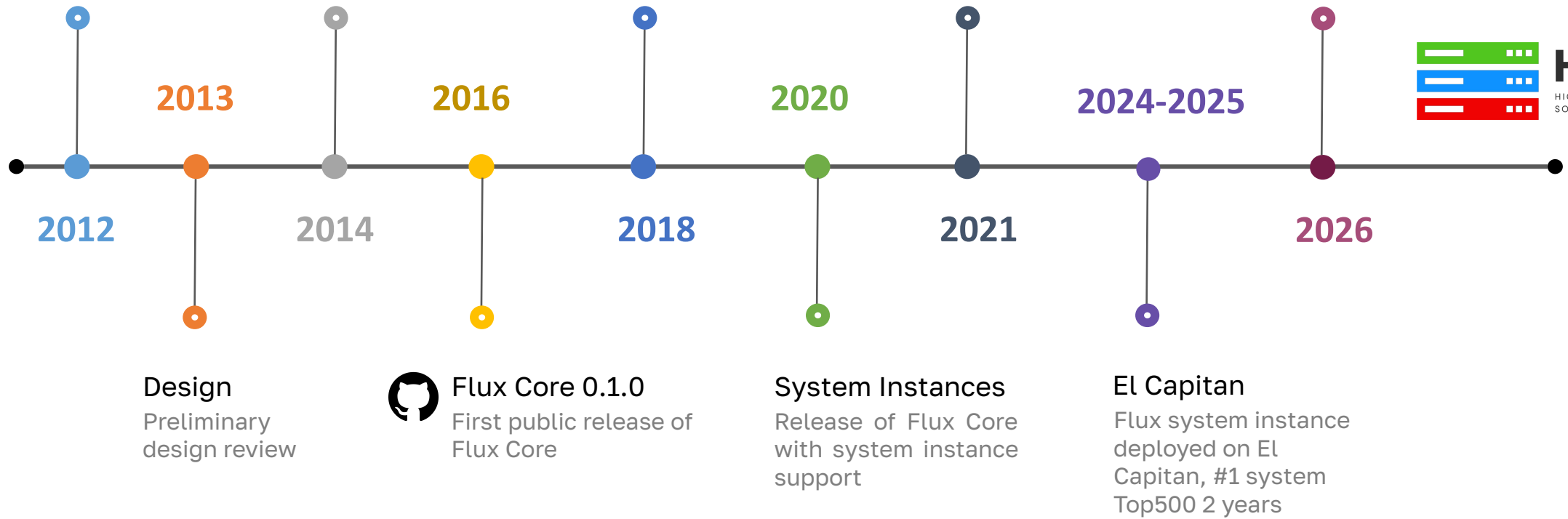
Project Established
Initial project funding
proposal and staffing

Instance Launch
Prototype Flux
instance that could
be launched by Slurm

Science
Flux used by Pilot2
cancer moonshot
ensemble

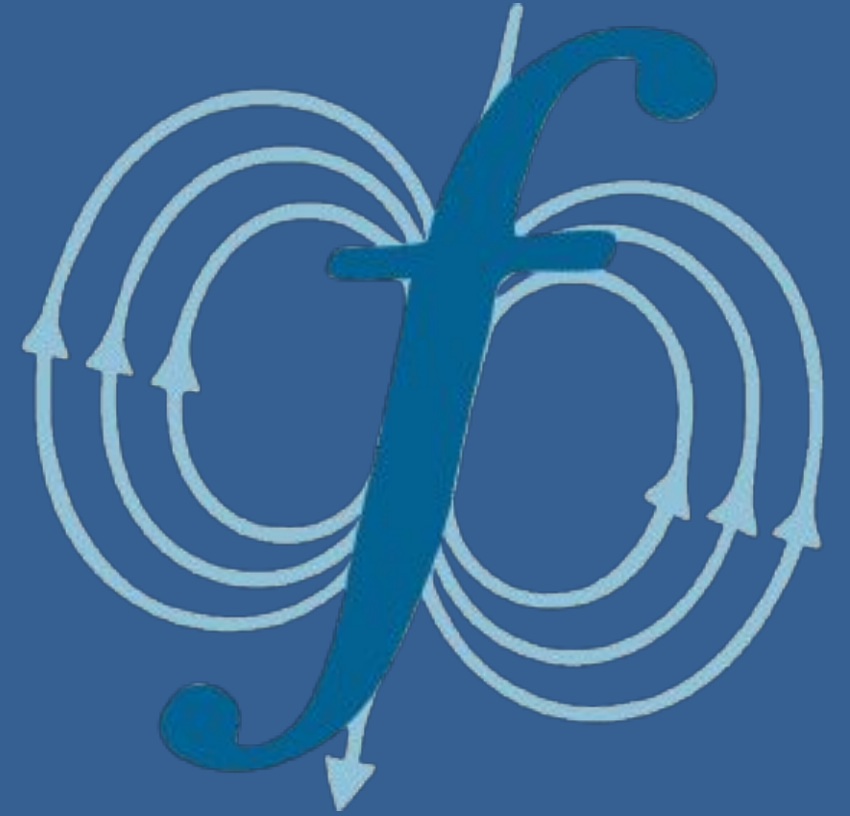
MuMMI Workflow
Flux+MuMMI
Integration used for
RAS-RAF membrane
dynamics ensemble

Community
Flux joins HPSF,
collaborations with
quantum, international,
take off.



What's new since last year

Highlights from a year of releases – August 2025 to today



Flux is hardened for the biggest machines



Faster, leaner core

- ✓ New modular boot system (modprobe) starts & stops Flux efficiently – now the default
- ✓ Broker modules can be built in and reloaded on the fly
- ✓ Zero-copy messaging for large transfers; threaded network overlay

Scale & resilience

- ✓ Streaming job lists and faster state dumps
- ✓ Job-aware filesystem check and repair (`flux fsck`)
- ✓ Resilient batch jobs survive loss of compute nodes
- ✓ System-instance hardening and recovery improvements

Flux is friendlier to use, easier to extend

Extend it in Python

- ✓ Write CLI plugins to add your own command options
- ✓ Broker and scheduler modules can now be written in Python
- ✓ A pure-Python resource model and a small “toy scheduler” to learn from

Nicer on the command line

- ✓ “Did you mean...?” suggestions when you mistype a command
- ✓ Full usernames by default, `NO_COLOR` support, wider flux jobs output
- ✓ New `flux cgroup`, `flux sproc`, and `flux config` commands



The Flux scheduler is still getting smarter

Smarter scheduling (Fluxion)

- ✓ Near-node “rabbit” storage as a first-class scheduled resource
- ✓ New resource-query engine: info, find, and cancel
- ✓ Flexible matching with `or_slot` and `XOR slot` — chooses among resource options
- ✓ Bulk property set/remove; amend resources inside sub-instances
- ✓ Grow and shrink endpoints supporting subgraphs of resources.



Flux continues to integrate with cloud

Kubernetes

- ✓ Native integration with Kubeflow Trainer for AI/ML Simulation
- ✓ fluence refactor: native gang scheduling and support for custom resources
- ✓ Flux Operator: support for PodGroup API, custom uid/gid, fsgroup and rootless execution.



Work underway, and future development

The Flux Scheduler (Fluxion)

- ✓ Better handling of expressing time, constraints, and elasticity.

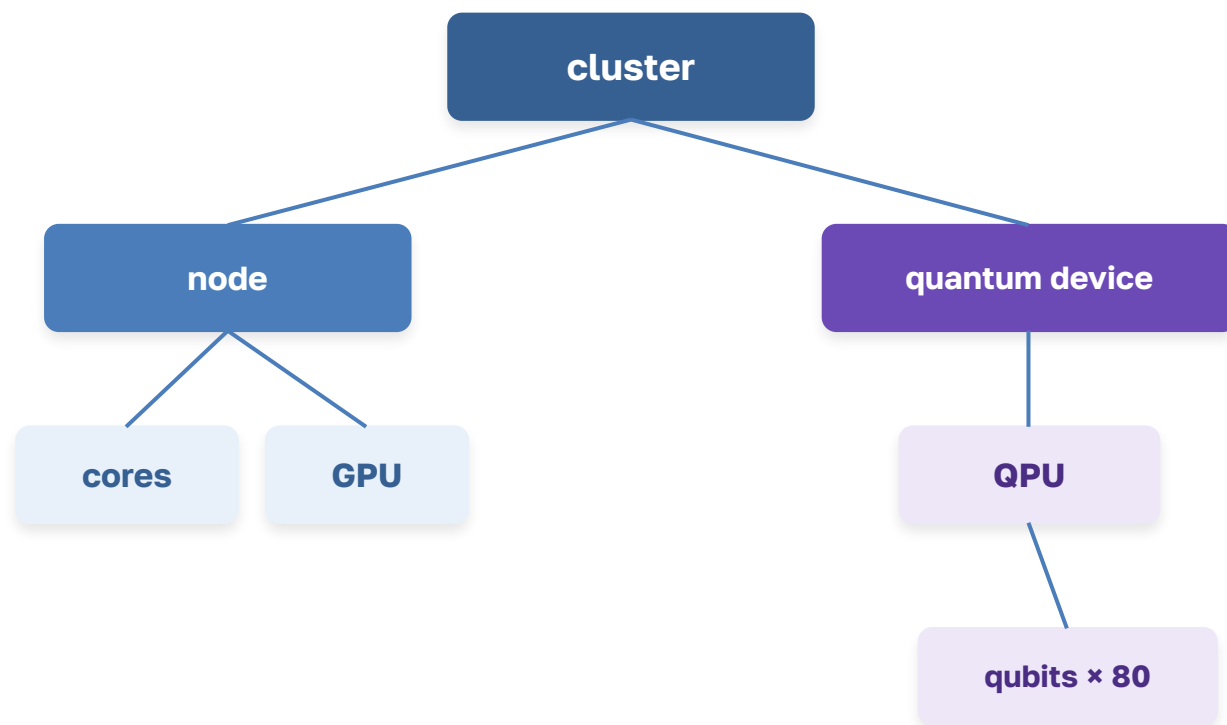
Custom Resources

- ✓ Integration and co-scheduling of custom resources like quantum.



Quantum is just another resource

Flux's graph-based scheduler was always meant to describe **resources that don't exist yet** and to handle intelligent co-scheduling. Quantum is the work-in-progress proof.



A job asks:

"give me 4 cores + 1 GPU + 20 qubits"

Same graph, new vertices

Fluxion sees the whole system as a graph. A quantum computer is just more vertices – device → QPU → qubits, each with attributes like backend or region.

Nothing is quantum-specific

The scheduler matches qubits exactly like it matches cores or GPUs. A resource "type" is just a label – so anything with a count and attributes can be scheduled.



github.com/converged-computing/fluence

Flux works well with agents

Our new library of MCP (model context protocol) tools makes it easy for agents to interact with their workload manager.



```
tools:
- path: hpc_mcp.timer.sleep_timer
- path: hpc_mcp.filesystem.filesystem_write_file
- path: hpc_mcp.filesystem.filesystem_read_file
- path: hpc_mcp.filesystem.filesystem_list_directory
- path: hpc_mcp.filesystem.filesystem_find_directory
- path: hpc_mcp.filesystem.filesystem_find_file
- path: hpc_mcp.build.spack.which_spack
- path: hpc_mcp.build.spack.spack_list
- path: hpc_mcp.build.spack.spack_find
- path: hpc_mcp.build.spack.spack_spec
- path: hpc_mcp.build.spack.spack_info
- path: hpc_mcp.build.spack.spack_install
- path: hpc_mcp.git.git_clone_tmp
- path: hpc_mcp.db.database_save
- path: hpc_mcp.db.database_get
- path: flux_mcp.resource.flux_resource_list
- path: flux_mcp.job.flux_submit_job
- path: flux_mcp.job.flux_cancel_job
- path: flux_mcp.job.flux_get_job_info
- path: flux_mcp.job.flux_get_job_logs
```



What's special about Flux?

Features that allow Flux to stand out as a workload manager



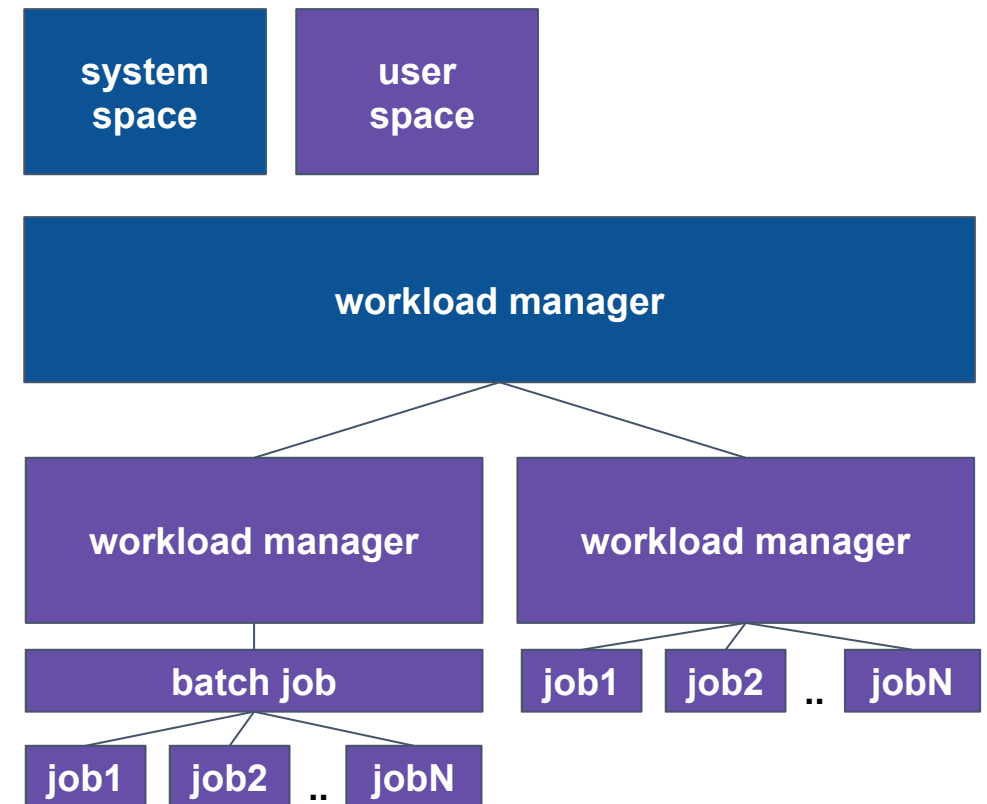
1. Emerging systems need a flexible resource model

- Resources like network, power, storage, and **<insert future resource>** are not easily integrated into existing schedulers.
- A *flexible resource model* can express arbitrary resources, including those that don't exist yet.



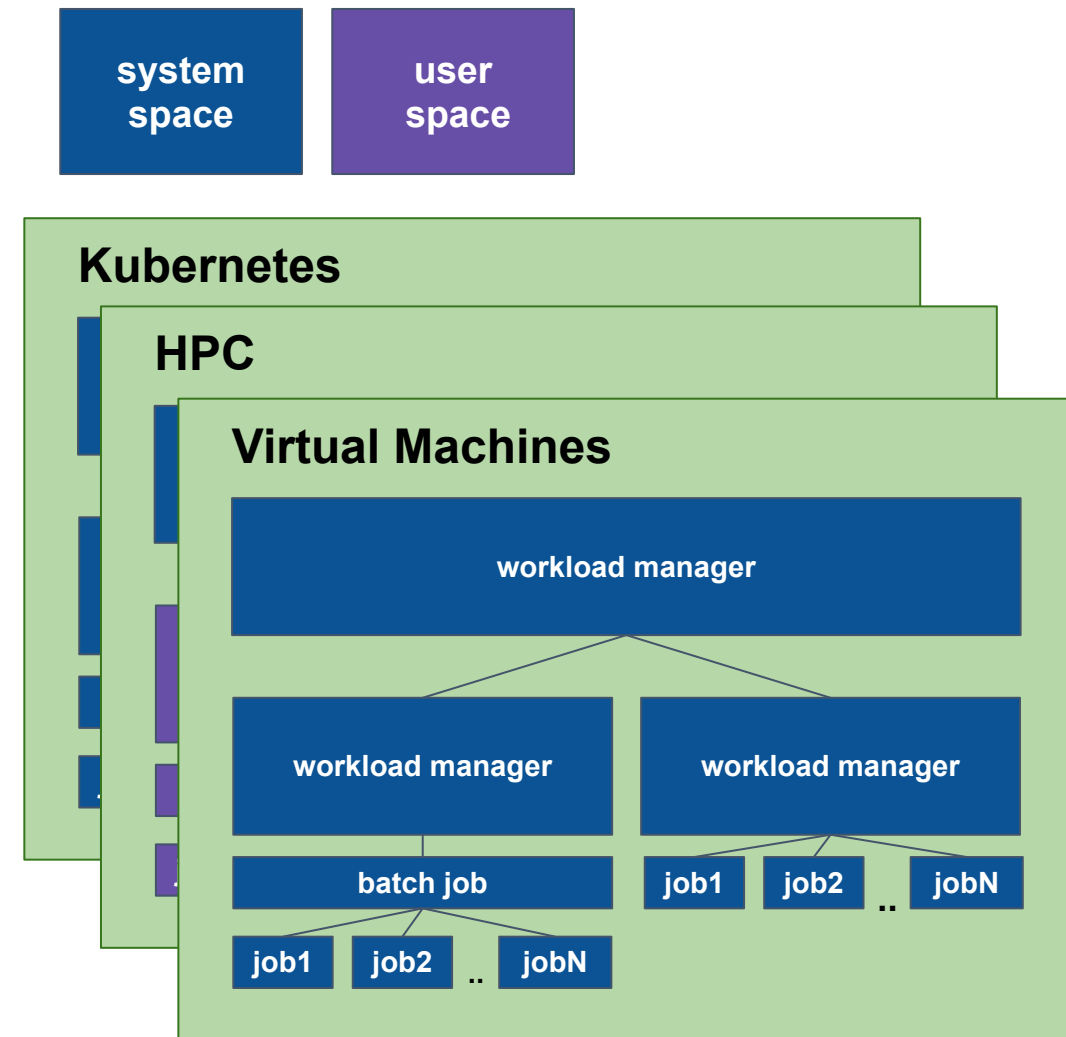
2. The hierarchical design facilitates portability

- A resource manager can be instantiated *as a job* under another resource manager
- The resource manager itself can be a portability layer.



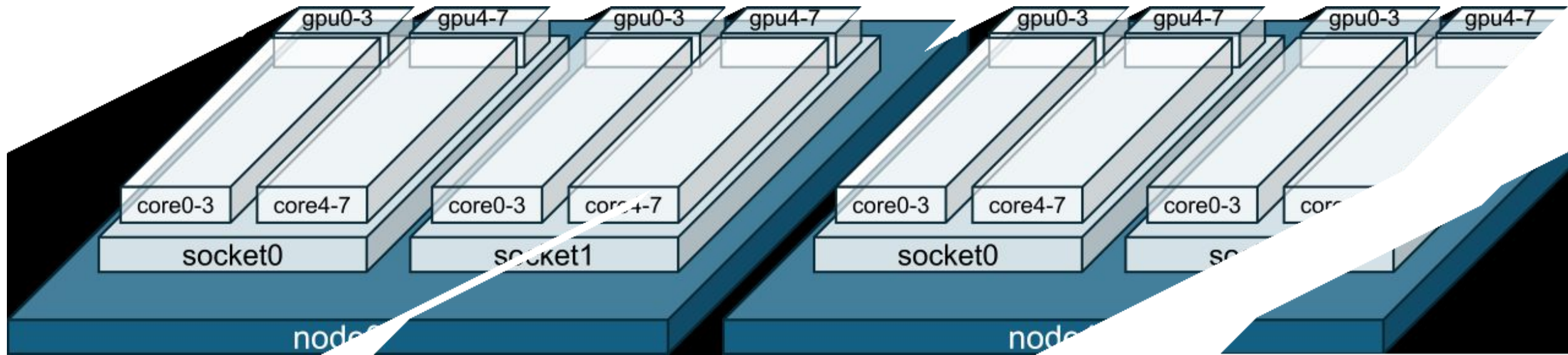
2. The hierarchical design facilitates portability

- A resource manager can be instantiated *as a job* under another resource manager
- The resource manager itself can be a portability layer.



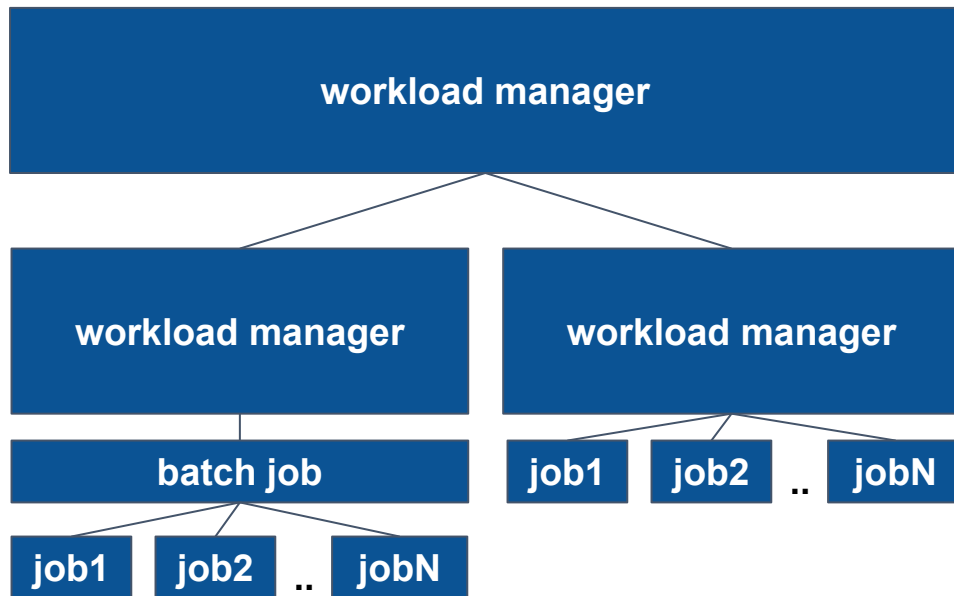
3. Hierarchical design facilitates flexibility, specificity

- **Scheduling decisions:** distributed at multiple levels.
- **Managing resources:** optimized for work at each level.
- **Throughput:** jobs can be processed at leaf nodes.



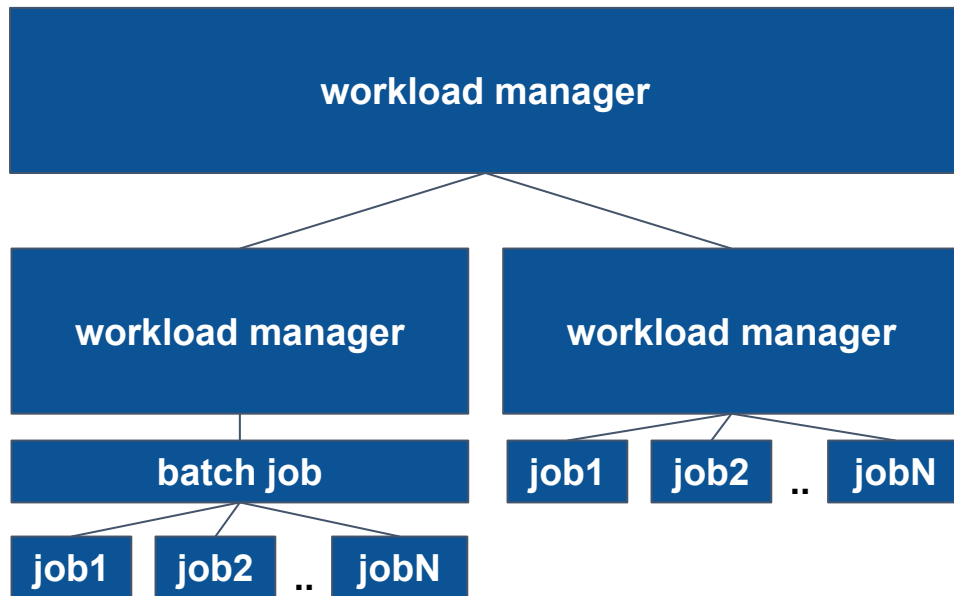
4. The design facilitates scheduler configurability

One scheduling algorithm to rule them all (what about queue)?

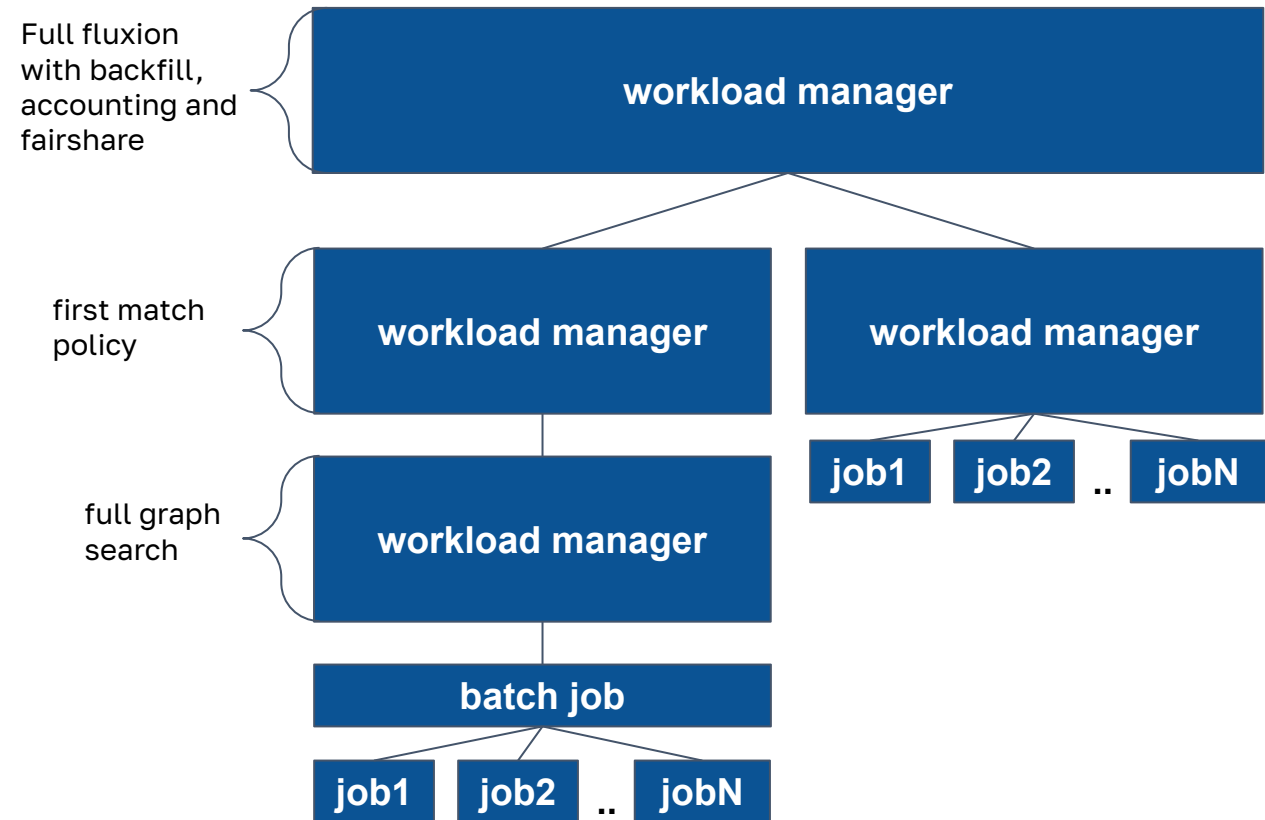


4. The design facilitates scheduler configurability

One scheduling algorithm to rule them all (what about queue)?

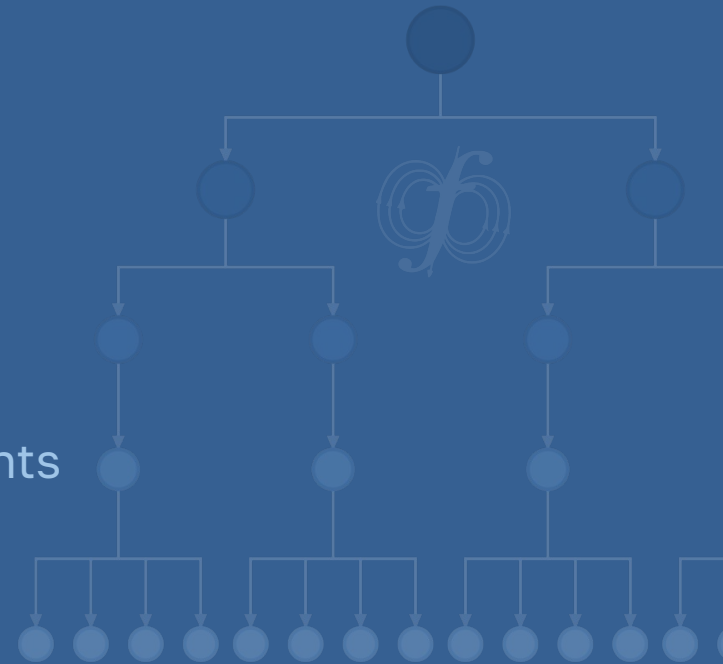


Choose an algorithm for each level (instance, or sub-graph) in the hierarchy.

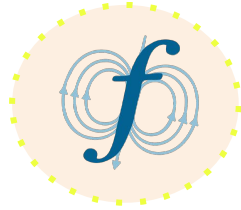


Flux architecture and projects

The core components of Flux, and how we extend them across environments

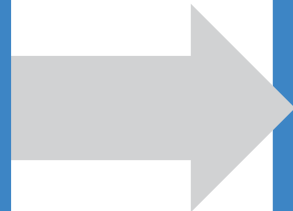


flux-core and flux-sched are the most important



flux-core

- 1. Command line client**
flux run ...
- 2. Job management**
transformation
validation
status
- 3. Messages and events**



flux-sched

- 1. Resource definition**
json, xml, etc.
- 2. Queueing**
- 3. Resource graph**
vertices (nodes, cores, socket, etc.)
edges "node1 contains socket1"
- 4. Query interface**

flux-sched models your system as a graph

flux-sched

1. Resource definition

json, xml, etc.

2. Queueing

3. Resource graph

vertices (nodes, cores, socket, etc.)
edges "node1 contains socket1"

4. Query interface

Resource hierarchy

Cluster

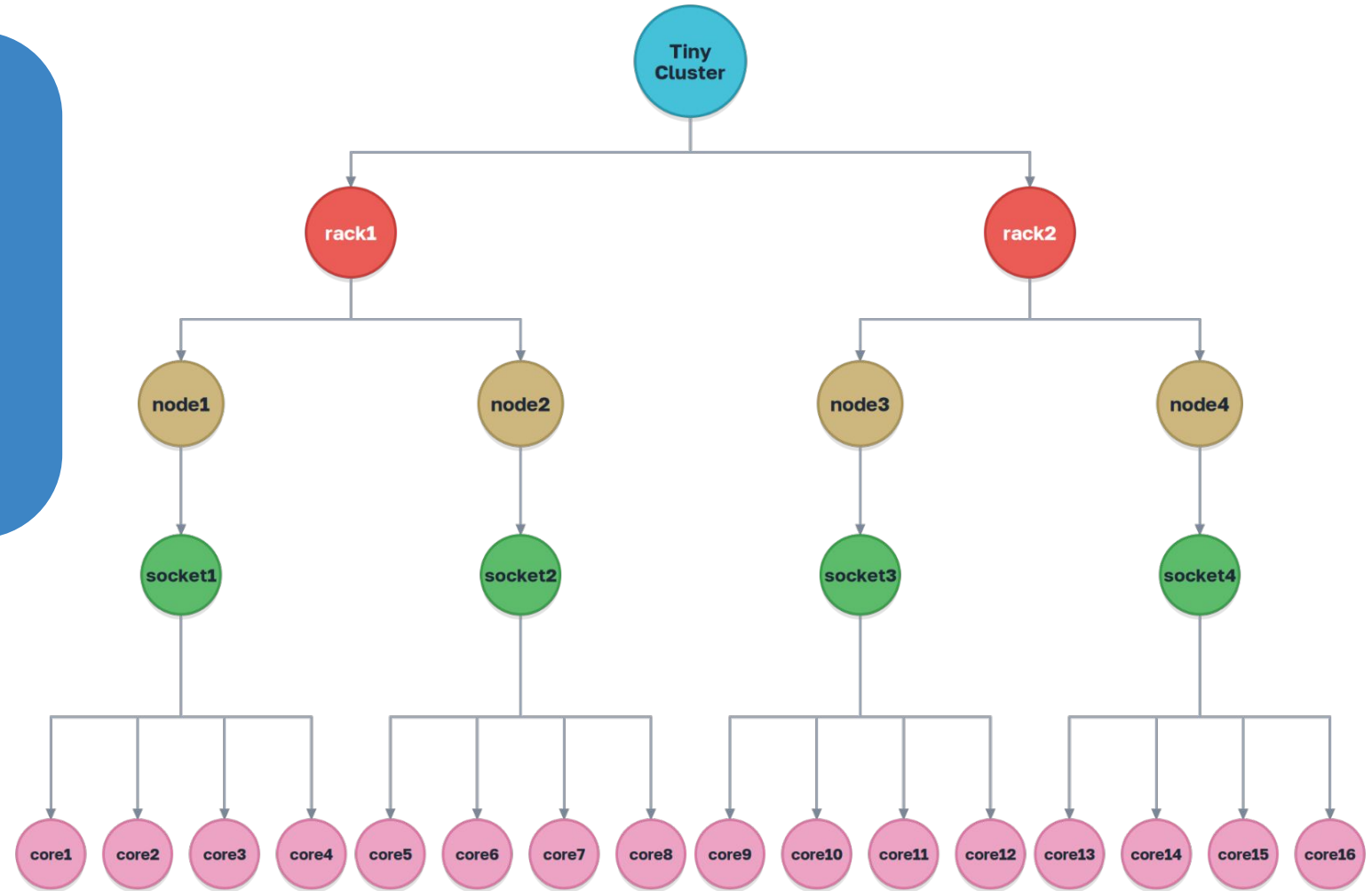
Rack

Node

Socket

Core

→ = CONTAINS



flux-core is what you interact with to manage jobs

flux-core

1. Command line client

flux run ...

2. Job management

transformation

validation

status

3. Messages and events

I want to:

-  Run a job with these resources
-  Watch and respond to job events
-  Cancel a job
-  Check my job status
-  Write a plugin to validate jobs

How do I request my application to run?

"I want 4 nodes, 384 cores"

```
flux run -N 4 -n 384 ./app
```

"Block terminal and stream output"

How do I request my application to run?

"I want 4 nodes, 384 cores"

```
flux run -N 4 -n 384 ./app
```

"Block terminal and stream output"

```
flux submit -N 4 -n 384 ./app
```

"The same, but not blocking"

How do I request my application to run?

"I want 4 nodes, 384 cores"

```
flux run -N 4 -n 384 ./app
```

"Block terminal and stream output"

```
flux submit -N 4 -n 384 ./app
```

"The same, but not blocking"

instance



How do I request my application to run?

"I want 4 nodes, 384 cores"

```
flux run -N 4 -n 384 ./app
```

"Block terminal and stream output"

```
flux submit -N 4 -n 384 ./app
```

"The same, but not blocking"

```
flux alloc -N 4 -n 384 bash
```

"Make me an interactive allocation"

How do I request my application to run?

"I want 4 nodes, 384 cores"

```
flux run -N 4 -n 384 ./app
```

"Block terminal and stream output"

```
flux submit -N 4 -n 384 ./app
```

"The same, but not blocking"

```
flux alloc -N 4 -n 384 bash
```

"Make me an interactive allocation"

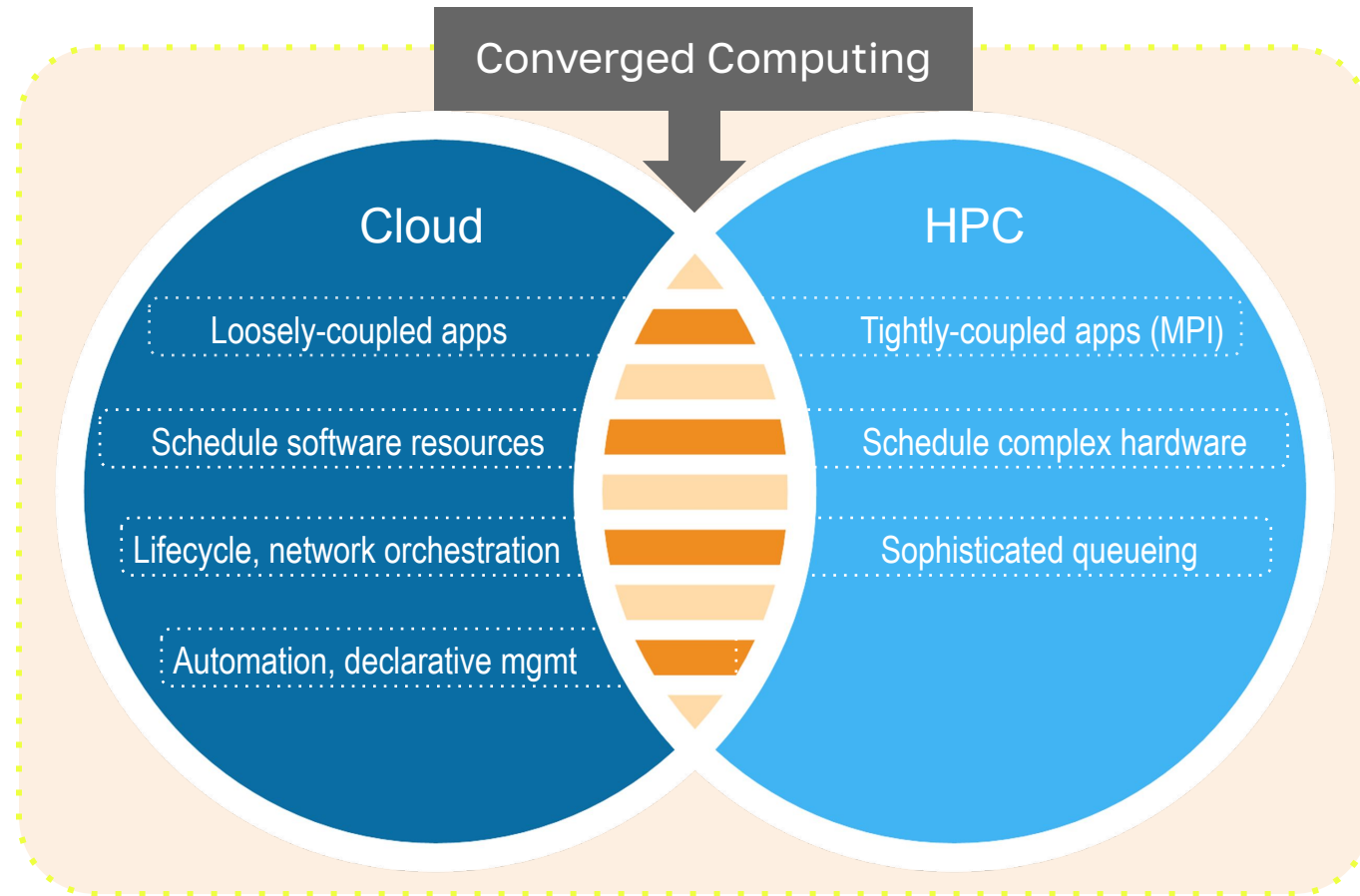
```
flux batch -N 4 -n 384 --wrap bash
```

"Make me a new instance, run script"

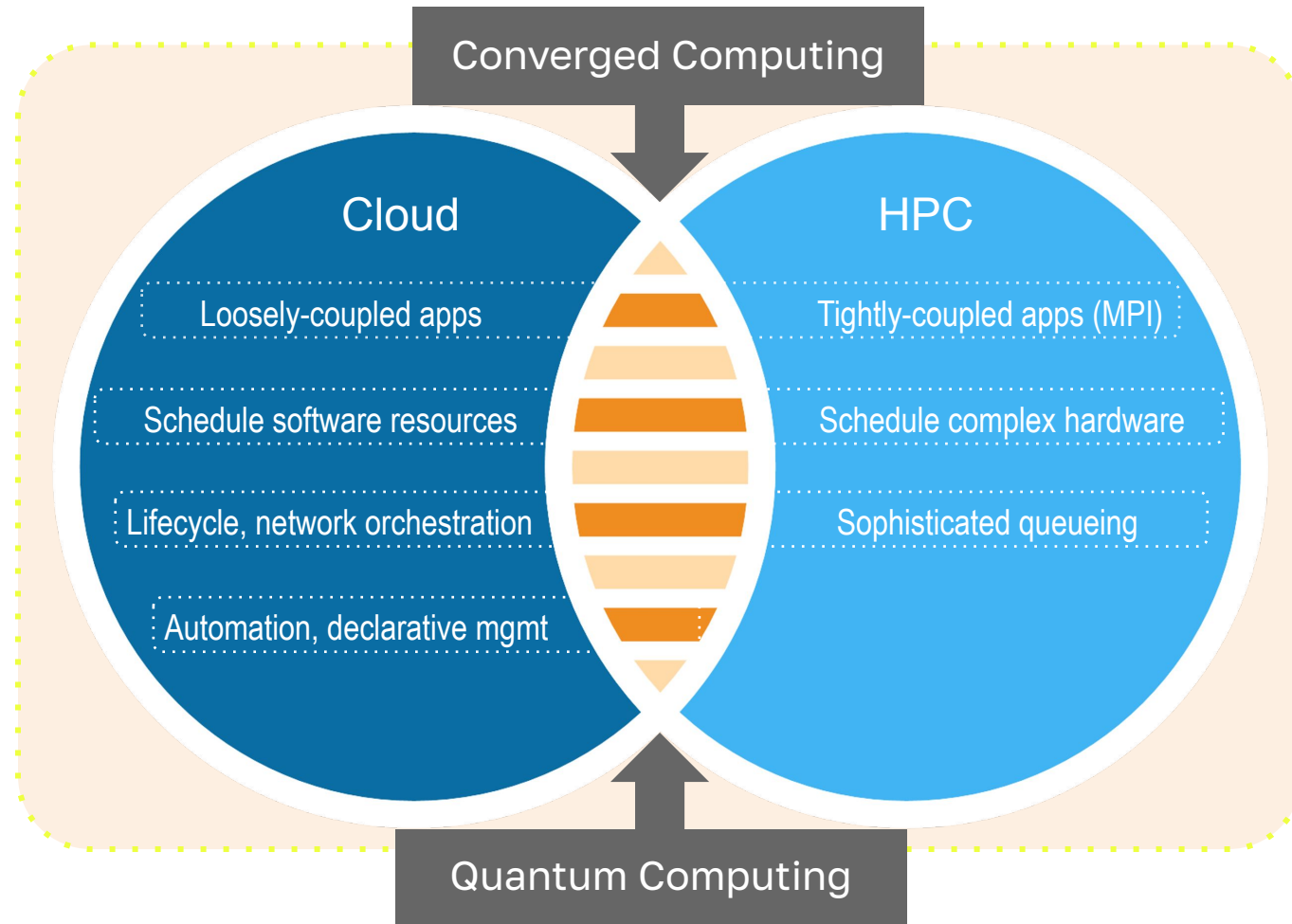
Flux enables converged computing

The best of both worlds between cloud and HPC, technology and culture

A best of both worlds between cloud-native and HPC



A best of both worlds between cloud-native and HPC



Portability is a new dimension of performance

If we want applications to be portable, which scenario makes sense?

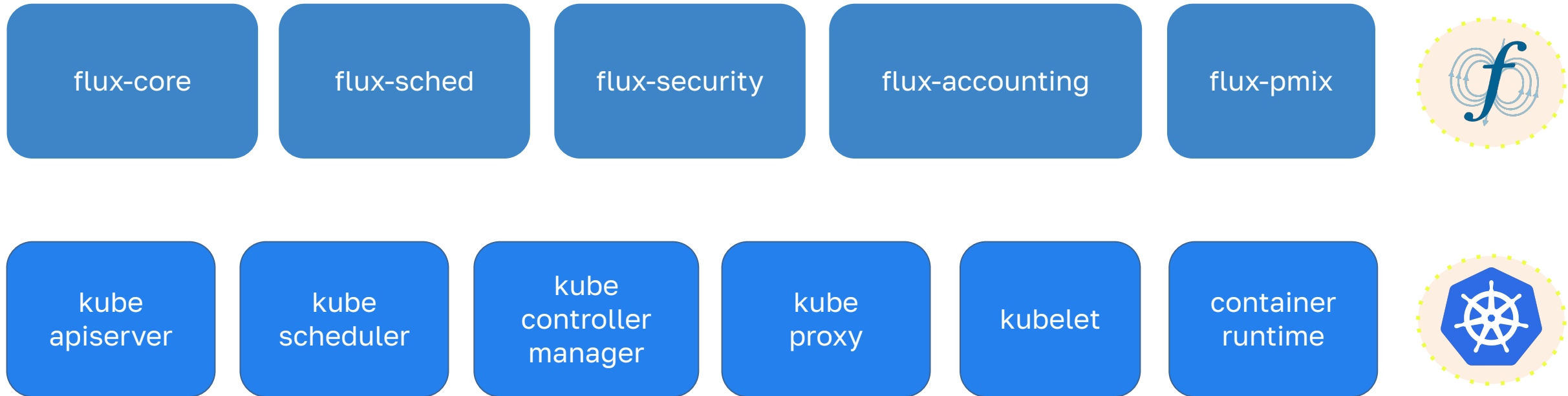


Optimize perfectly for one environment.



Build very good for many environments

Kubernetes and Flux are analogous...



Both are modular – composed of interchangeable components!

The Kubeflow Trainer

Simulation with AI/ML

Kubeflow + Flux for AI/ML Simulation

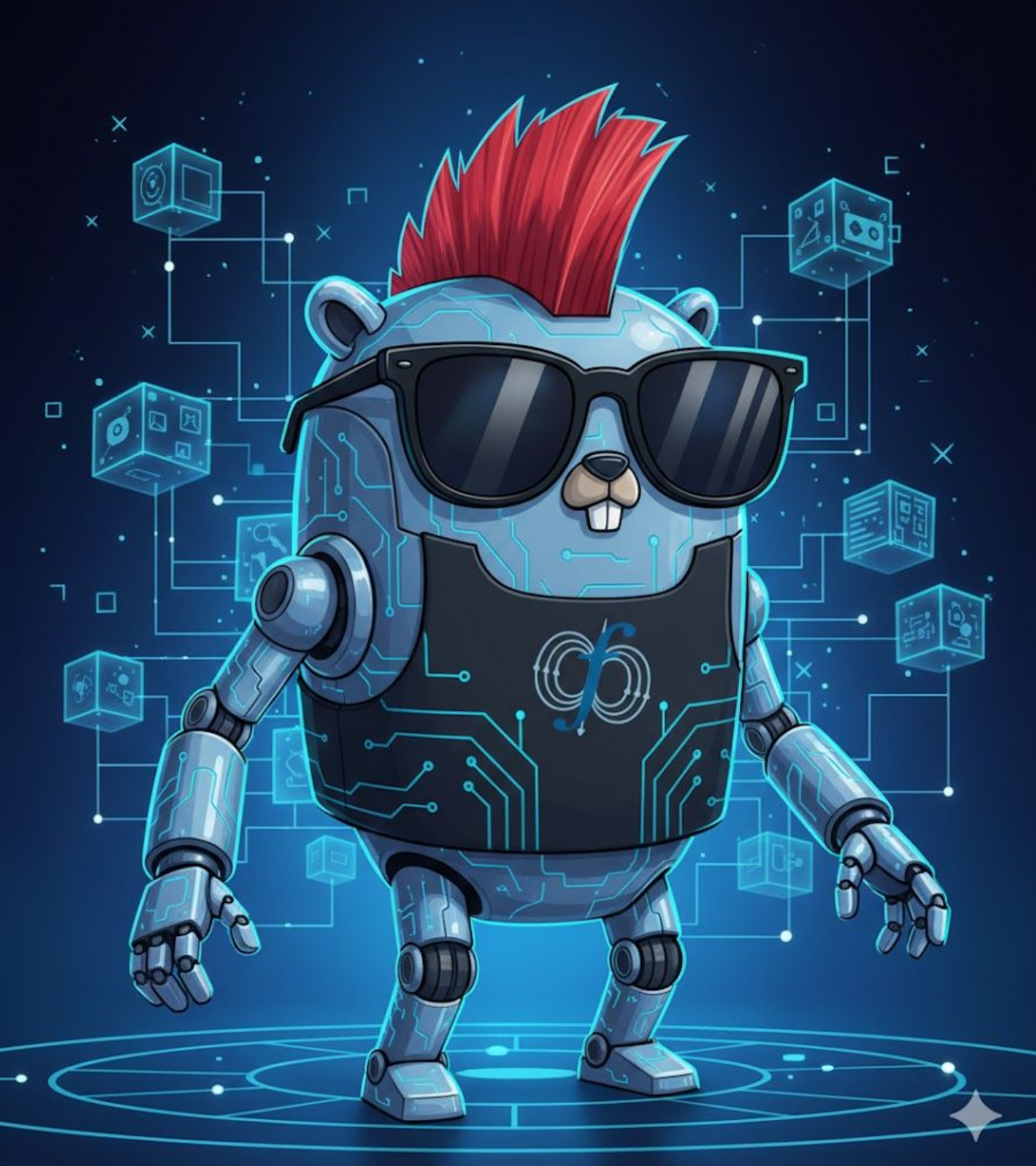
Low-latency communication for MPI simulation paired with surrogate models



- **Kubeflow Trainer:** Industry standard for AI/ML training
- Traditional scientific simulations are often informed by AI/ML models (GPUs)
- Physics-based simulation or scientific workloads are often CPU-based MPI tasks
- Current Kubeflow MPI support was limited in MPI variants and ssh bootstrap

The Flux Operator

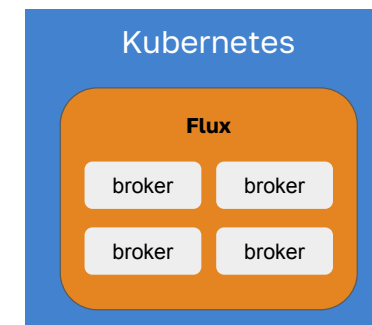
Deploy an HPC cluster in Kubernetes



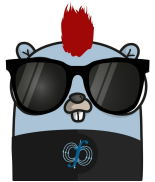
The Flux Operator

Deploys Flux in Kubernetes

We map Flux components into Kubernetes abstractions to implement the entirety of Flux inside of Kubernetes via an *operator*, a controller that knows how to manage objects in a Kubernetes cluster.



An entire MiniCluster is created via a YAML definition



MiniCluster Definition

container: <application>
command: <command>
size: <nodes>

Reconcile

Kubernetes

follower

leader

follower

follower

follower

follower

follower

follower

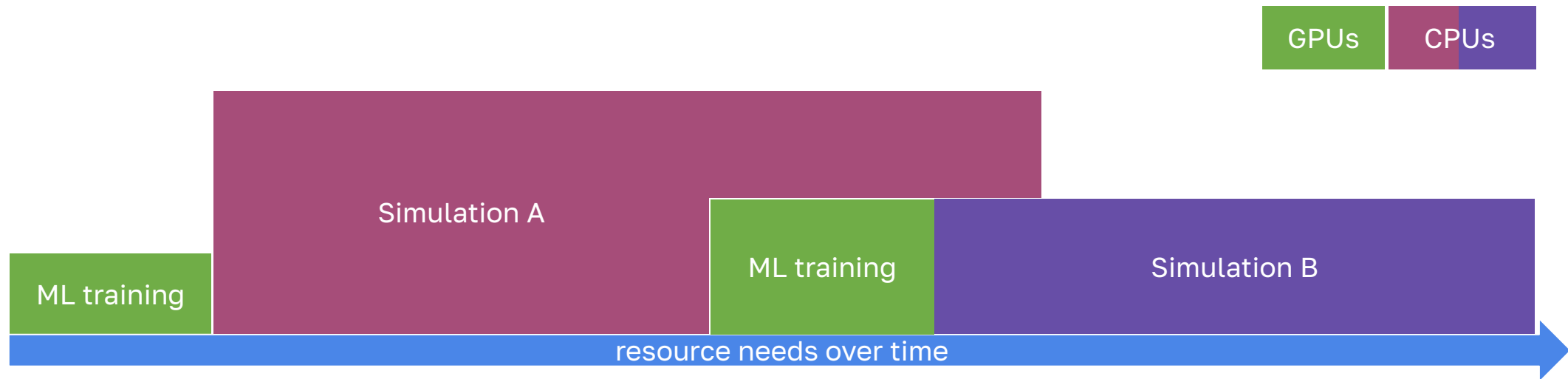
follower

Flux MiniCluster



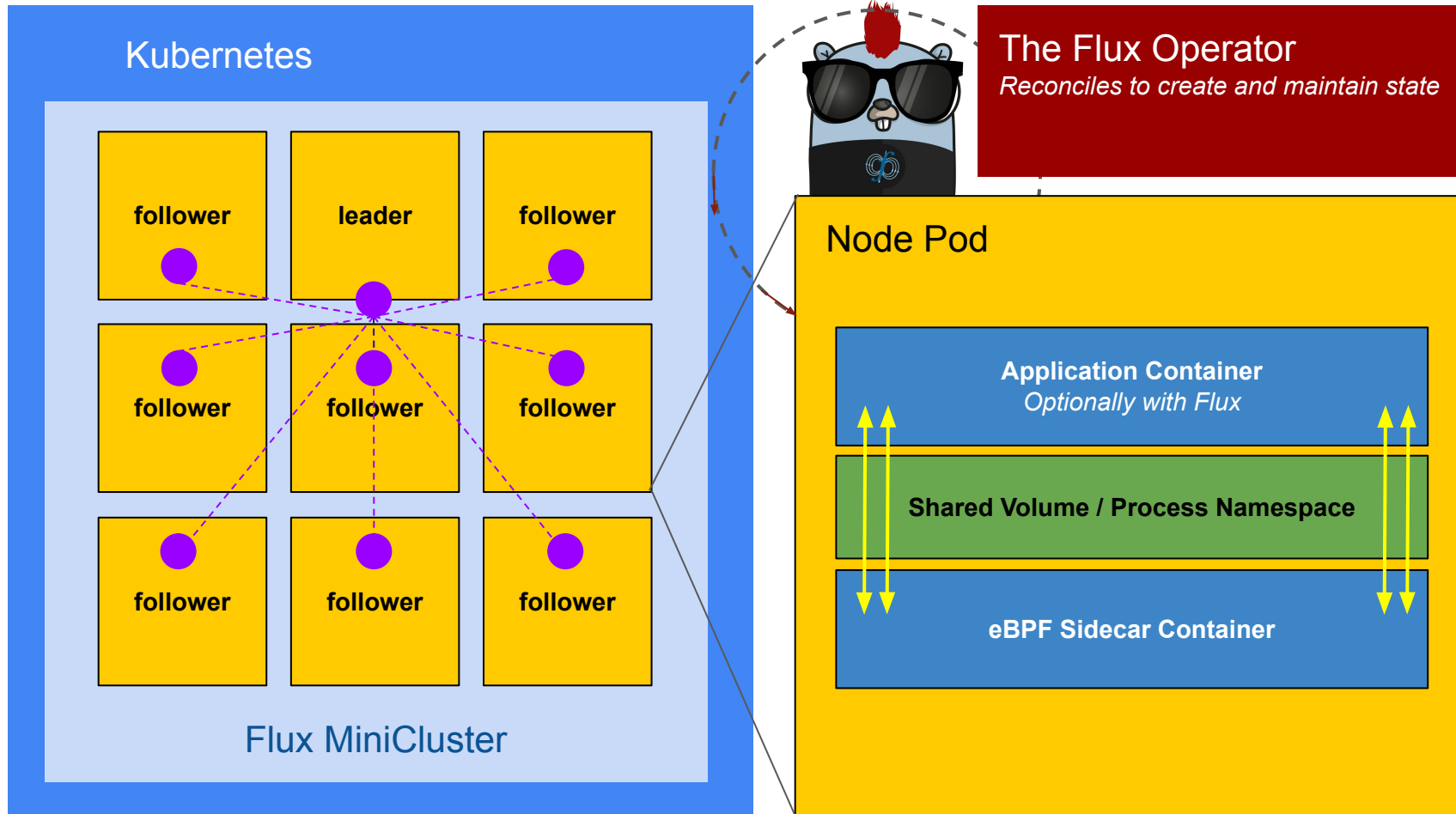
The Flux Operator has supported elasticity since 2023

- Designed to manage the complex, dynamic workloads required by coupled AI/ML and simulation.



A dynamic, HPC cluster in cloud that does not require over-provisioning of resources.

We can collect performance metrics with eBPF



fluence

Gang scheduling, custom resources

Fluence: the user interface

"I want to co-schedule quantum resources (qpus) for my classical job..."

What the user writes

```
# quantum-pod.yaml
schedulerName: fluence
resources:
  limits:
    fluxion.flux-framework.org/qpu: "1"
```

No group. No hard-coded backend — the webhook + PreBind supply it.

```
$ kubectl logs sampler -f
submitting sampler job to
  ibm_marrakesh
done: 2070 bytes from
  ibm_marrakesh
```

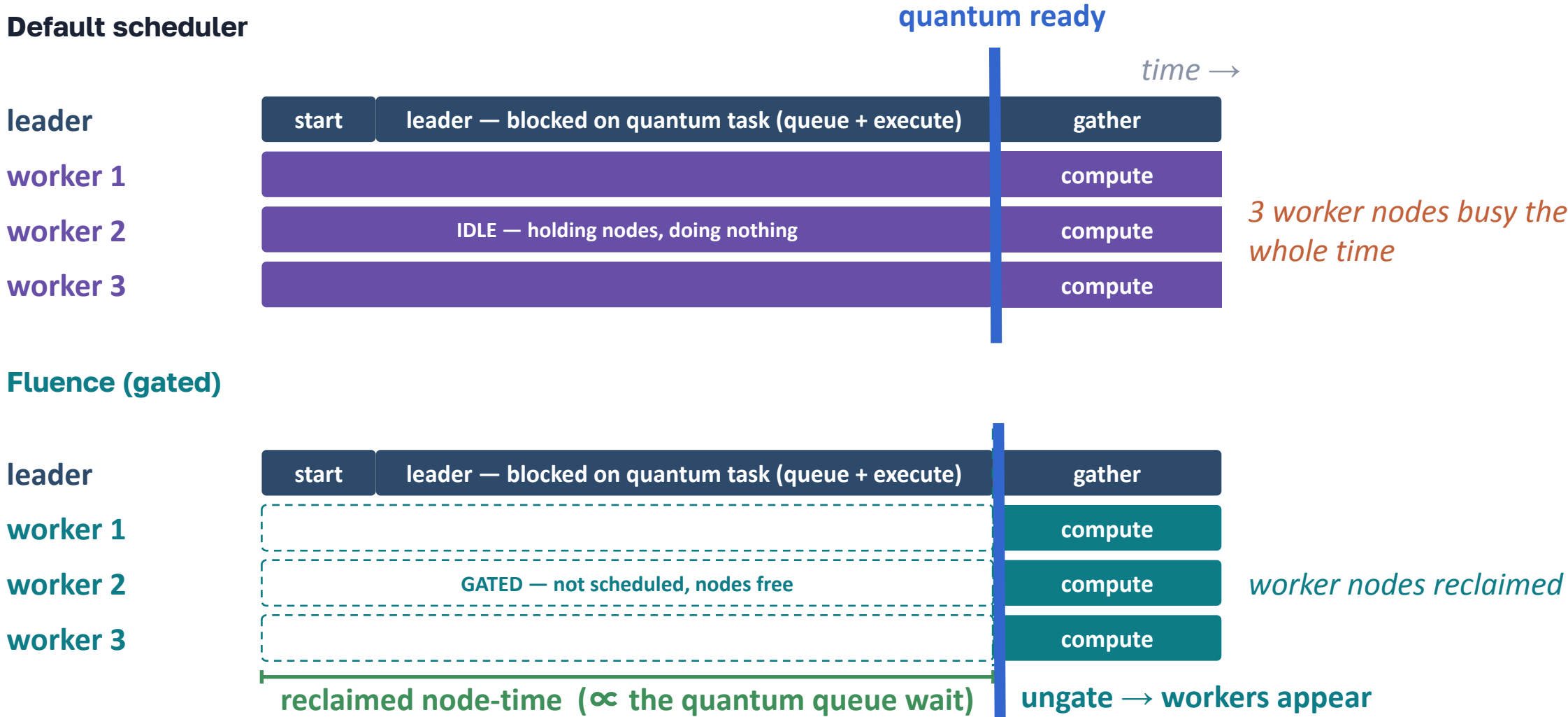
 Flux Framework and
Quantum: Early Work

Talk and Demos on YouTube



Fluence addresses the two-queue problem

Intelligent gating and monitoring to ensure classical and quantum start at the same time



Let's start the engines! 🚀

What will we be doing today?

- Using our EC2 Spawner
 - <https://tutorial.flux-framework.org>
 - Choose a unique username (if not, you'll be sharing an EC2 node)
 - Password: **HPCIC-2026-olive-oil**



Choose your own adventure! 🤠

- **Chapter 1:** Introduction to Flux
I am new to Flux and want to learn the basics.
- **Chapter 2:** Flux Python
I am using Flux with workflow tools / in Python.
- **Chapter 3:** Flux Plumbing
I want to learn niche, interesting tricks!
- **Chapter 4:** Flux with User-space Kubernetes
I want to run user-space Kubernetes and Flux with AI/ML workloads.
- **Chapter 5:** Flux with Quantum *new!*
I want to interact with quantum devices from my classical machine.

<https://tutorial.flux-framework.org>

<your choice of unique username>

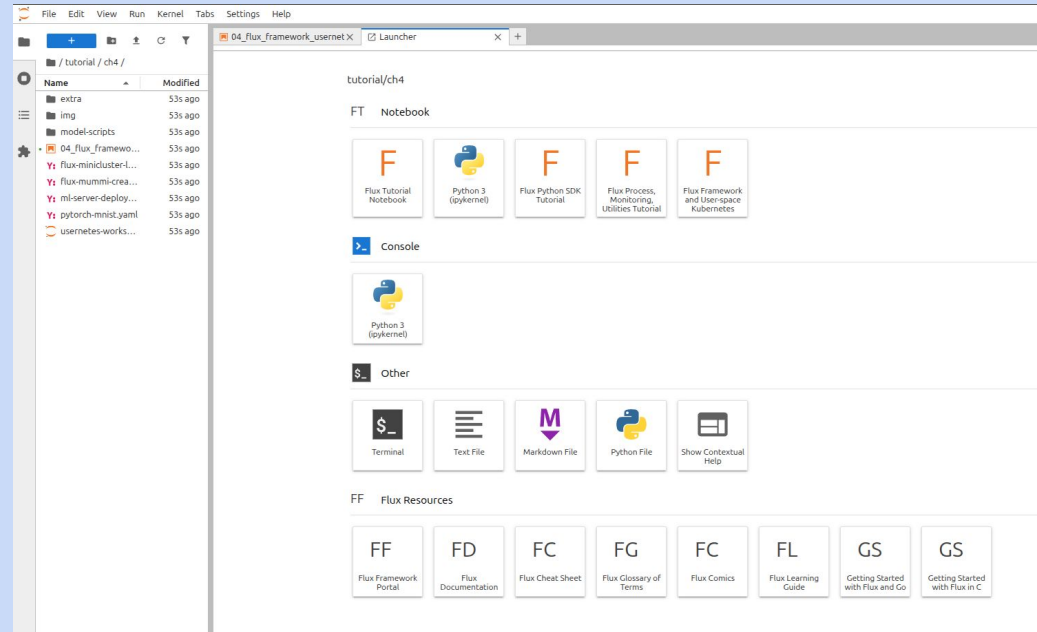
HPCIC-2026-olive-oil

JupyterHub EC2 Spawner

JupyterHub
hpc7g.16xlarge



```
flux start --test-size=4 jupyterlab ..  
hpc7g.16xlarge
```



Amazon
EC2

<https://tutorial.flux-framework.org>
<your choice of unique username>
HPCIC-2026-olive-oil

JupyterHub EC2 Spawner

JupyterHub
hpc7g.16xlarge



```
flux start jupyterlab ..  
hpc7g.16xlarge
```

flux-node-0
64 cores



Amazon
EC2

<https://tutorial.flux-framework.org>
<your choice of unique username>
HPCIC-2026-olive-oil

JupyterHub EC2 Spawner

JupyterHub
hpc7g.16xlarge



```
flux start -test-size=4 jupyterlab ..  
hpc7g.16xlarge
```

flux-node-0

flux-node-0

flux-node-0

flux-node-0



Amazon
EC2

<https://tutorial.flux-framework.org>
<your choice of unique username>
HPCIC-2026-olive-oil

JupyterHub EC2 Spawner

JupyterHub
hpc7g.16xlarge



```
ubuntu@ip-10-0-27-8:~/tutorial/ch4$ flux resource list
```

STATE	NNODES	NCORES	NGPUS	NODELIST
free	4	256	0	ip-10-0-27-[8,8,8,8]
allocated	0	0	0	
down	0	0	0	



Amazon
EC2

```
flux start -test-size=4 jupyterlab ..  
hpc7g.16xlarge
```

flux-node-0

flux-node-0

flux-node-0

flux-node-0

<https://tutorial.flux-framework.org>
<your choice of unique username>
HPCIC-2026-olive-oil

JupyterHub EC2 Spawner

JupyterHub
hpc7g.16xlarge



```
flux start -test-size=4 jupyterlab ..  
hpc7g.16xlarge
```

flux-node-0

flux-node-0

flux-node-0

flux-node-0

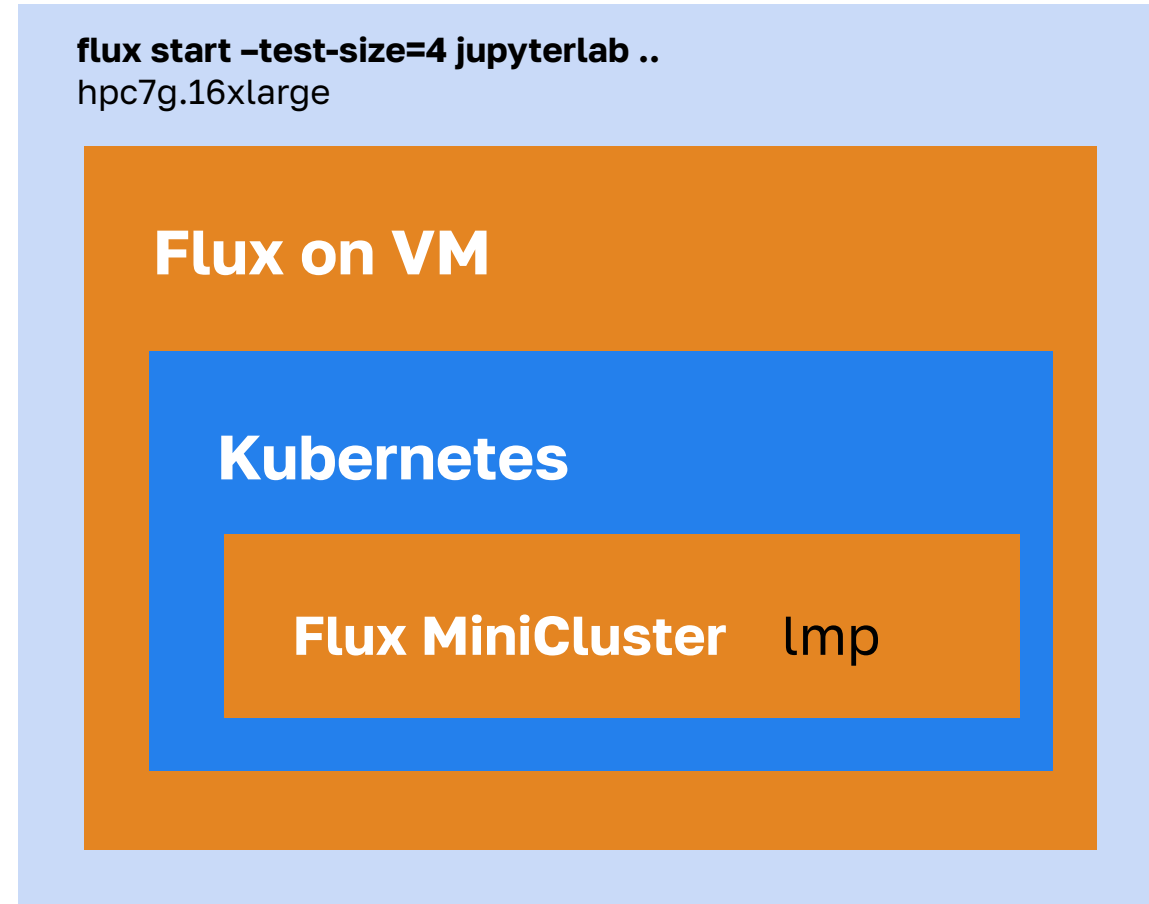
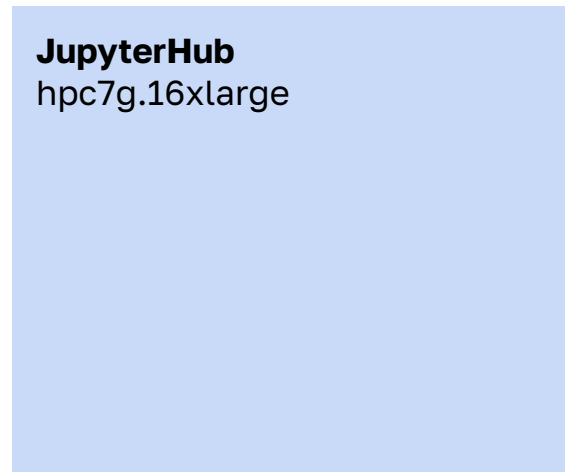
control plane



Amazon
EC2

<https://tutorial.flux-framework.org>
<your choice of unique username>
HPCIC-2026-olive-oil

LAMMPS in Flux, in Kubernetes, in Flux!



<https://tutorial.flux-framework.org>
<your choice of unique username>
HPCIC-2026-olive-oil

A quick primer on Kubernetes...

- 🥑 **kubectl** "kube cuttle" or "kube control" is how we interact with the cluster
- 🥑 **YAML** files are applied or created to create abstractions
- 🥑 A **Pod** is the smallest unit of abstraction - a holder for containers and resource set
- 🥑 A **Job** creates pods that have state (Running through Completed)

<https://tutorial.flux-framework.org>
<your choice of unique username>
HPCIC-2026-olive-oil

Accessing the hands-on tutorial...

- Using our EC2 Spawner
 - <https://tutorial.flux-framework.org>
 - Choose a unique username (if not, you'll be sharing an EC2 node)
 - Password: **HPCIC-2026-olive-oil**
- We'd like your feedback! Please take our tutorial survey:
<https://converged-computing.org/community-survey>
Important: no ending slash above!



Thank you!

sochat1@llnl.gov | milroy1@llnl.gov

